# CastorTS: Causality-Guided Siamese Pretraining Model for Efficient Time Series Forecasting Appendix

# I PRELIMINARIES

## A. *State Space Model (SSM)*

The classical State Space Models (SSMs) describe the state evolution of a linear time-invariant system, which maps an input signal $\mathbf{x} = \{x(1), \cdots, x(t), \cdots\} \in \mathbb{R}^{L \times D} \mapsto \mathbf{y} = \{y(1), \cdots, y(t), \cdots\} \in \mathbb{R}^{L \times D}$ through implicit latent state $h(t) \in \mathbb{R}^{N \times D}$, where $t$, $L$, $D$ and $N$ indicate the time step, sequence length, channel number of the signal and state size, respectively. These models can be formulated as the following linear ordinary differential equations:

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t) + \mathbf{D}x(t), \tag{I.1}$$

where $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the state transition matrix that describes how states change over time, $\mathbf{B} \in \mathbb{R}^{N \times D}$ is the input matrix that controls how inputs affect state changes, $\mathbf{C} \in \mathbb{R}^{N \times D}$ denotes the output matrix that indicates how outputs are generated based on current states and $\mathbf{D} \in \mathbb{R}^{D}$ represents the command coefficient that determines how inputs affect outputs directly. Most SSMs exclude the second term in the observation equation, i.e., set $\mathbf{D}x(t) = 0$, which can be recognized as a skip connection in deep learning models. The time-continuous nature poses challenges for integration into deep learning architectures. To alleviate this issue, most methods utilize the Zero-Order Hold rule [1] to discretize continuous time into $K$ intervals, which assumes that the function value remains constant over the interval $\mathbf{\Delta} \in \mathbb{R}^{D}$. The Eq. (I.1) can be reformulated as:

$$h_t = \overline{\mathbf{A}}h_{t-1} + \overline{\mathbf{B}}x_t, \quad y_t = \mathbf{C}h_t, \tag{I.2}$$

where $\overline{\mathbf{A}} = \exp(\mathbf{\Delta A})$ and $\overline{\mathbf{B}} = (\mathbf{\Delta A})^{-1}(\exp(\mathbf{\Delta A}) - \mathbf{I}) \cdot \mathbf{\Delta B}$. Discrete SSMs can be interpreted as a combination of CNNs and RNNs. Typically, the model employs a convolutional mode for efficient, parallelizable training and switches to a recurrent mode for efficient autoregressive inference. The formulations in Eq. (I.2) are equivalent to the following convolution [2]:

$$\begin{aligned} &\overline{\mathbf{K}} = (\mathbf{C}\overline{\mathbf{B}}, \mathbf{C}\overline{\mathbf{A}\mathbf{B}}, \cdots, \mathbf{C}\overline{\mathbf{A}}^{k}\overline{\mathbf{B}}, \cdots), \\ &\mathbf{y} = \mathbf{x} * \overline{\mathbf{K}}, \end{aligned} \tag{I.3}$$

Thus, the overall process can be represented as:

$$\mathbf{y} = \mathbf{SSM}(\overline{\mathbf{A}}, \overline{\mathbf{B}}, \mathbf{C})(\mathbf{x}). \tag{I.4}$$

## B. *Selective State Space Model (Mamba)*

The discrete SSMs are based on data-independent parameters, meaning that parameters $\bar{\mathbf{A}}$, $\bar{\mathbf{B}}$, and $\mathbf{C}$ are time-invariant and the same for any input, limiting their effectiveness in compressing context into a smaller state [1]. Mamba [1] introduces a selective mechanism to filter out extraneous information while retaining pertinent details indefinitely. Specifically, it utilizes Linear Projection to parameterize the weight matrices $\{\mathbf{B}_t\}_{t=1}^{L}$, $\{\mathbf{C}_t\}_{t=1}^{L}$ and $\{\mathbf{\Delta}_t\}_{t=1}^{L}$ according to model input $\{x_t\}_{t=1}^{L}$, improving the context-aware ability, i.e.,:

$$\begin{aligned} &\overline{\mathbf{B}}_t = \texttt{Linear}_{\mathbf{B}}(x_t), \\ &\overline{\mathbf{C}}_t = \texttt{Linear}_{\mathbf{C}}(x_t), \\ &\mathbf{\Delta}_t = \texttt{Softplus}\left(\texttt{Linear}_{\mathbf{\Delta}}(x_t)\right). \end{aligned} \tag{I.5}$$

Then the output sequence $\{y_t\}_{i=t}^{L}$ can be computed with those input-adaptive discretized parameters as follows:

$$h_t = \overline{\mathbf{A}}_t h_{t-1} + \overline{\mathbf{B}}_t x_t, \quad y_t = \overline{\mathbf{C}_t} h_t. \tag{I.6}$$

The selection mechanism hinders SSMs from supporting parallel training like CNNs. To overcome this, Mamba adopts Parallel Associative Scan [3] and Memory Recomputation. The former exploits linear associativity and the parallelism of GPUs and TPUs for memory-efficient computation, while the latter reduces memory usage by recomputing intermediate states during backpropagation.

## C. *Mamba-2*

Mamba-2 [4] introduces a comprehensive framework known as Structured State-Space Duality (SSD). Leveraging SSD, it reformulates SSMs as semi-separable matrices via matrix transformation and develops a more hardware-efficient computation

method based on block-decomposed matrix multiplication, i.e.,

$$\begin{aligned} \mathbf{y} &= \mathbf{SSD}(\mathbf{A}, \mathbf{B}, \mathbf{C})(\mathbf{x}) \\ &= \mathbf{M}\mathbf{x} \\ \text{s.t. } \mathbf{M} &= \begin{pmatrix} \mathbf{C}_0^\top \mathbf{A}_{0:0} \mathbf{B}_0 & & & \\ \mathbf{C}_1^\top \mathbf{A}_{1:0} \mathbf{B}_0 & \mathbf{C}_1^\top \mathbf{A}_{1:1} \mathbf{B}_1 & & \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{C}_j^\top \mathbf{A}_{j:0} \mathbf{B}_0 & \mathbf{C}_j^\top \mathbf{A}_{j:1} \mathbf{B}_1 & \cdots & \mathbf{C}_j^\top \mathbf{A}_{j:i} \mathbf{B}_i \end{pmatrix} \end{aligned} \tag{I.7}$$

where $\mathbf{M}$ denotes the matrix form of SSMs that uses the sequentially semiseparable representation, and $\mathbf{M}_{ji} = \mathbf{C}_j^\top \mathbf{A}_{j:i} \mathbf{B}_i$, $\mathbf{C}_j$ and $\mathbf{B}_i$ represent the selective space state matrices associated with input tokens $x_j$ and $x_i$, respectively. $\mathbf{A}_{j:i}$ denotes the selective matrix of hidden states corresponding to the input tokens ranging from $j$ to $i$. Mamba-2 achieves a 2-8$\times$ faster training process than Mamba-1's parallel associative scan while remaining competitive with Transformers.

## II Additional Details on CastorTS

### A. *Siamese Pair Sampler*

We introduce a Siamese Pair Sampler (SPS) to improve sample efficiency and encourage cross-domain common-dynamics learning under a limited pretraining budget. Instead of treating each time series independently, SPS constructs paired inputs from different domains, forcing the model to compare heterogeneous temporal patterns and identify transferable regularities.

Let the open-domain pretraining corpus be $\mathcal{D} = \{\mathcal{D}^1, \mathcal{D}^2, \ldots, \mathcal{D}^I\}$, where $\mathcal{D}^i$ contains $M^i$ samples. A sample from domain $i$ is denoted as $\mathbf{X}_i \in \mathbb{R}^{T \times N^i}$, where $N^i$ is the number of variates and may vary across domains. At each pretraining step, SPS first samples an anchor $\mathbf{X}_i$ from domain $i$, and then samples a counterpart $\mathbf{X}_j$ from another domain $j \neq i$, forming a Siamese pair $(\mathbf{X}_i, \mathbf{X}_j)$. Compared with instance-level pretraining, SPS substantially enlarges the effective training space. While standard pretraining uses at most $\sum_{i=1}^{I} M^i$ samples, ordered cross-domain pairing can generate up to

$$\sum_{i=1}^{I} \sum_{\substack{j=1 \\ j \neq i}}^{I} M^i M^j \tag{II.1}$$

distinct pairs. This increases sample diversity and provides explicit cross-domain contrast, helping the model separate domain-specific variations from transferable common dynamics.

### B. *Dynamic Tokenization vs. Fixed Patch Tokenization*

Fixed patch tokenization, as used in PatchTST [5], divides a time series into uniform windows of length $P$. Although simple and efficient, it imposes the same temporal granularity on the whole sequence. This is suboptimal for non-stationary time series, where coherent transitions may be split across adjacent patches and distinct temporal regimes may be merged into one patch. DyToken addresses this limitation through content-adaptive tokenization. Instead of using a predefined patch size, it detects token boundaries according to representation changes, as defined in Eq. (9)–(10). It then applies EMA-based smoothing in Eq. (11) and boundary-aligned downsampling in Eq. (12), producing variable-length tokens that adapt to local temporal dynamics.

This design brings three benefits. First, DyToken allocates more tokens to informative regions such as turning points and regime shifts, while compressing long stationary intervals. Second, it reduces redundant tokens and improves robustness to domain-wise temporal heterogeneity, including different sampling rates, seasonal amplitudes, and regime durations. Third, it produces explicit boundary cues $\{p_t^{(l)}, b_t^{(l)}\}$, which are reused in Stage 3 for dynamic de-tokenization. Therefore, DyToken enables boundary-consistent coarse-to-fine reconstruction, rather than relying on heuristic upsampling.

### C. *Stage 3: Temporal Decoding and Dynamic De-Tokenization.*

Stage 3 inversely mirrors Stage 1 to progressively restore the original temporal resolution for full-granularity reconstruction. It takes the coarsest tokens $\mathbf{X}^{(L+1)}$ output by Stage 2 and initializes $\mathbf{X}^{(L)} \leftarrow \mathbf{X}^{(L+1)}$. Then, for $l = L-1, \ldots, 0$, we alternately apply $\text{De-DyToken}^{(l)}(\cdot)$ and $\text{SSD-Decoder}^{(l)}(\cdot)$:

$$\begin{aligned} \tilde{\mathbf{X}}^{(l)} &= \text{De-DyToken}^{(l)}\big(\mathbf{X}^{(l+1)}; \mathbf{p}^{(l)}, \mathbf{b}^{(l)}\big), \\ \mathbf{X}^{(l)} &= \text{SSD-Decoder}^{(l)}\Big(\tilde{\mathbf{X}}^{(l)}\Big). \end{aligned} \tag{II.2}$$

We apply the following operations to each variate segment and omit the variate index for simplicity.

*(a) Dynamic de-tokenization via* $\mathrm{De\text{-}DyToken}(\cdot)$. At level $l$, Stage 1 provides boundary cues $\{p_t^{(l)}, b_t^{(l)}\}_{t=1}^{T_l}$ computed by Eq. (9) and the thresholding rule Eq. (10), and we enforce $b_1^{(l)} = 1$. The boundary index set is

$$\mathcal{B}^{(l)} = \{t \mid b_t^{(l)} = 1\} \subseteq \{1, \dots, T_l\}, \tag{II.3}$$

so the coarse tokens at level $l+1$ correspond to boundary positions selected in Stage 1. We write

$$\mathbf{X}^{(l+1)} = \{\mathbf{x}_j^{(l+1)}\}_{j=1}^{T_{l+1}} \in \mathbb{R}^{T_{l+1} \times D}, \qquad T_{l+1} = \sum_{t=1}^{T_l} b_t^{(l)}, \tag{II.4}$$

where $j$ indexes boundary tokens in temporal order, i.e., the $j$-th token corresponds to the $j$-th element of $\mathcal{B}^{(l)}$.

*Downsampled boundary confidence.* To align confidence weights with coarse tokens, we downsample $\{p_t^{(l)}\}_{t \in \mathcal{B}^{(l)}}$ to obtain

$$\mathbf{P}^{(l+1)} = \{P_j^{(l+1)}\}_{j=1}^{T_{l+1}} \in [0,1]^{T_{l+1}}, \qquad P_j^{(l+1)} = p_{t_j}^{(l)}, \quad t_j \in \mathcal{B}^{(l)}, \tag{II.5}$$

where $t_j$ is the boundary index associated with the $j$-th coarse token.

*EMA smoothing on coarse tokens.* Following the dechunking-style interpolation [6], we smooth adjacent coarse tokens with an EMA driven by $\mathbf{P}^{(l+1)}$:

$$\bar{\mathbf{x}}_j^{(l+1)} = P_j^{(l+1)}\, \mathbf{x}_j^{(l+1)} + \big(1 - P_j^{(l+1)}\big)\bar{\mathbf{x}}_{j-1}^{(l+1)}, \qquad j = 2, \dots, T_{l+1}, \tag{II.6}$$

with $\bar{\mathbf{x}}_1^{(l+1)} = \mathbf{x}_1^{(l+1)}$.

*Causal expansion to length $T_l$.* We then expand smoothed coarse tokens back to the fine-grained timeline. Define the cumulative boundary index

$$s_t^{(l)} = \sum_{k=1}^{t} b_k^{(l)} \in \{1, \dots, T_{l+1}\}, \qquad t = 1, \dots, T_l, \tag{II.7}$$

and expand by indexing the latest boundary token up to time $t$:

$$\tilde{\mathbf{x}}_t^{(l)} = \bar{\mathbf{x}}_{s_t^{(l)}}^{(l+1)}, \qquad t = 1, \dots, T_l. \tag{II.8}$$

This yields $\tilde{\mathbf{X}}^{(l)} = \{\tilde{\mathbf{x}}_1^{(l)}, \dots, \tilde{\mathbf{x}}_{T_l}^{(l)}\} \in \mathbb{R}^{T_l \times D}$. Stacking $N$ variate segments gives $\tilde{\mathbf{X}}^{(l)} \in \mathbb{R}^{(NT_l) \times D}$.

*STE-gated boundary-consistency weighting.* To keep boundary learning end-to-end differentiable, we further gate the expanded features with a boundary-consistency weight [6]:

$$c_t^{(l)} = \big(p_t^{(l)}\big)^{b_t^{(l)}} \big(1 - p_t^{(l)}\big)^{1 - b_t^{(l)}}, \ \mathrm{STE}(c) = c + \mathrm{stopgrad}(1 - c), \tag{II.9}$$

$$\tilde{\mathbf{x}}_t^{(l)} \leftarrow \mathrm{STE}\big(c_t^{(l)}\big) \cdot \tilde{\mathbf{x}}_t^{(l)}, \qquad t = 1, \dots, T_l. \tag{II.10}$$

*(b) Temporal decoding via* $\mathrm{SSD\text{-}Decoder}(\cdot)$. Given $\tilde{\mathbf{X}}^{(l)} \in \mathbb{R}^{(NT_l) \times D}$, we refine it with $\mathrm{SSD\text{-}Decoder}^{(l)}(\cdot)$, which shares the same Mamba/SSD-style architecture as the encoder in Stage 1. For brevity, we write

$$\mathbf{X}^{(l)} = \mathrm{SSD\text{-}Decoder}^{(l)}\Big(\tilde{\mathbf{X}}^{(l)}\Big). \tag{II.11}$$

*Overall recursion.* By iterating $\mathrm{De\text{-}DyToken}^{(l)}(\cdot)$ and $\mathrm{SSD\text{-}Decoder}^{(l)}(\cdot)$ from $l = L-1$ down to 0, Stage 3 progressively restores the temporal resolution and outputs the full-granularity representation $\mathbf{X}^{(0)}$ for temporal-level reconstruction objectives.

## III Theorem Proof

### A. *Proof of Floor Attention Theorem*

**Theorem III.1.** *(Lower-bound guarantee for proxy dominance) Assume each* $\mathrm{FloorAttn}(\cdot)$ *layer uses Eq.* (15) *with the same* $\lambda \in (0,1)$*, so that* $\widehat{\mathbf{A}}_{i,0}^{(h)} \geq \lambda$ *for all* $i$ *and all layers* $h$*. Further assume the proxy token is* absorbing*, i.e.,* $\widehat{\mathbf{A}}_{0,0}^{(h)} = 1$ *and* $\widehat{\mathbf{A}}_{0,j}^{(h)} = 0$ *for all* $j \geq 1$*. Let* $\mathbf{\Pi}^{(H)} = \widehat{\mathbf{A}}^{(H)}\widehat{\mathbf{A}}^{(H-1)} \cdots \widehat{\mathbf{A}}^{(1)}$*. Then for any token* $i \in \{0, 1, \dots, N\}$ *and any depth* $H \geq 1$*,*

$$\mathbf{\Pi}_{i,0}^{(H)} \geq 1 - (1 - \lambda)^H. \tag{III.1}$$

*Consequently,* $\sum_{j=1}^{N} \mathbf{\Pi}_{i,j}^{(H)} \leq (1-\lambda)^H$ *for all* $i$*, and for any* $j \in \{1, \dots, N\}$*,*

$$\mathcal{I}_j^{(H)} \triangleq \sum_{i=0}^{N} \mathbf{\Pi}_{i,j}^{(H)} \leq (N+1)(1-\lambda)^H, \ \mathcal{I}_0^{(H)} \geq (N+1)\big(1 - (1-\lambda)^H\big). \tag{III.2}$$

*In particular, if*

$$H \geq H_{\min} \triangleq \left\lceil \frac{\log(1/2)}{\log(1-\lambda)} \right\rceil, \tag{III.3}$$

*then* $\mathcal{I}_0^{(H)} \geq \mathcal{I}_j^{(H)}$ *for all* $j \in \{1, \ldots, N\}$.

*Proof.* Fix any layer $h$. Since $\widehat{\mathbf{A}}_{i,0}^{(h)} \geq \lambda$ and each row sums to 1, we have

$$\sum_{j=1}^{N} \widehat{\mathbf{A}}_{i,j}^{(h)} = 1 - \widehat{\mathbf{A}}_{i,0}^{(h)} \leq 1 - \lambda, \qquad \forall i. \tag{III.4}$$

Let $\mathcal{S} = \{0, 1, \ldots, N\}$ and $\bar{\mathcal{S}} = \{1, \ldots, N\}$. For any row probability vector $\mathbf{p} \in \mathbb{R}^{N+1}$, define its total *non-proxy weight* $\|\mathbf{p}\|_{\bar{\mathcal{S}}} \triangleq \sum_{j \in \bar{\mathcal{S}}} p_j$. We claim that applying one layer contracts the non-proxy weight by at most $(1-\lambda)$:

$$\|\mathbf{p}\widehat{\mathbf{A}}^{(h)}\|_{\bar{\mathcal{S}}} \leq (1-\lambda)\|\mathbf{p}\|_{\bar{\mathcal{S}}}. \tag{III.5}$$

To see this, expand

$$\|\mathbf{p}\widehat{\mathbf{A}}^{(h)}\|_{\bar{\mathcal{S}}} = \sum_{j \in \bar{\mathcal{S}}} \sum_{i \in \mathcal{S}} p_i \, \widehat{\mathbf{A}}_{i,j}^{(h)} = \sum_{i \in \mathcal{S}} p_i \Big( \sum_{j \in \bar{\mathcal{S}}} \widehat{\mathbf{A}}_{i,j}^{(h)} \Big).$$

By Eq. (III.4), the inner sum is at most $(1-\lambda)$ for all $i \geq 1$. For $i = 0$, the absorbing-proxy assumption implies $\sum_{j \in \bar{\mathcal{S}}} \widehat{\mathbf{A}}_{0,j}^{(h)} = 0$. Therefore,

$$\|\mathbf{p}\widehat{\mathbf{A}}^{(h)}\|_{\bar{\mathcal{S}}} \leq (1-\lambda) \sum_{i=1}^{N} p_i = (1-\lambda)\|\mathbf{p}\|_{\bar{\mathcal{S}}},$$

proving Eq. (III.5).

Now fix a token $i$ and define the rollout row distribution after $H$ layers:

$$\mathbf{p}^{(H)} \triangleq \mathbf{e}_i^\top \mathbf{\Pi}^{(H)} = \mathbf{e}_i^\top \widehat{\mathbf{A}}^{(H)} \cdots \widehat{\mathbf{A}}^{(1)}.$$

Applying Eq. (III.5) repeatedly gives

$$\|\mathbf{p}^{(H)}\|_{\bar{\mathcal{S}}} \leq (1-\lambda)^H \|\mathbf{e}_i^\top\|_{\bar{\mathcal{S}}} \leq (1-\lambda)^H.$$

Since each row of $\mathbf{\Pi}^{(H)}$ sums to 1,

$$\mathbf{\Pi}_{i,0}^{(H)} = 1 - \sum_{j=1}^{N} \mathbf{\Pi}_{i,j}^{(H)} = 1 - \|\mathbf{p}^{(H)}\|_{\bar{\mathcal{S}}} \geq 1 - (1-\lambda)^H,$$

which proves Eq. (III.1). The inequality $\sum_{j=1}^{N} \mathbf{\Pi}_{i,j}^{(H)} \leq (1-\lambda)^H$ follows immediately.

Summing the bound over $i$ yields

$$\mathcal{I}_0^{(H)} = \sum_{i=0}^{N} \mathbf{\Pi}_{i,0}^{(H)} \geq (N+1)\big(1 - (1-\lambda)^H\big),$$

and for any $j \geq 1$,

$$\mathcal{I}_j^{(H)} = \sum_{i=0}^{N} \mathbf{\Pi}_{i,j}^{(H)} \leq \sum_{i=0}^{N} \sum_{k=1}^{N} \mathbf{\Pi}_{i,k}^{(H)} \leq (N+1)(1-\lambda)^H,$$

proving Eq. (III.2).

Finally, if $(1-\lambda)^H \leq \frac{1}{2}$ (equivalently $H \geq H_{\min}$ in Eq. (III.3)), then

$$\mathcal{I}_0^{(H)} \geq (N+1)\Big(1 - \tfrac{1}{2}\Big) = \tfrac{N+1}{2} \geq (N+1)(1-\lambda)^H \geq \mathcal{I}_j^{(H)},$$

for all $j \in \{1, \ldots, N\}$, completing the proof. □

### *B. Proof of Hyperspherical Theorem*

**Theorem III.2** (Scale invariance and stability of hyperspherical proxies)**.** *Let* $g : \mathbb{R}^d \to \mathbb{R}^m$ *be the proxy projector and define* $u(z) = g(z)/\|g(z)\|_2 \in \mathbb{S}^{m-1}$ *with* $\|g(z)\|_2 > 0$. *Assume two domains produce proxy tokens related by* $z' = \alpha z + \delta$ *with*

$\alpha > 0$. *(i) If* $\delta = \mathbf{0}$*, then* $u(z') = u(z)$ *and cosine similarity is invariant to* $\alpha$*. (ii) If* $\|\delta\|_2 < \|g(\alpha z)\|_2$*, then*

$$\left\|u(z') - u(\alpha z)\right\|_2 \leq \frac{2\|g(z') - g(\alpha z)\|_2}{\|g(\alpha z)\|_2 - \|g(z') - g(\alpha z)\|_2}, \tag{III.6}$$

*and if* $g$ *is* $L_g$*-Lipschitz, it further holds that*

$$\left\|u(z') - u(\alpha z)\right\|_2 \leq \frac{2L_g\|\delta\|_2}{\|g(\alpha z)\|_2 - L_g\|\delta\|_2}. \tag{III.7}$$

*Proof.* (i) (Scale invariance) For any $\alpha > 0$,

$$u(\alpha z) = \frac{g(\alpha z)}{\|g(\alpha z)\|_2}. \tag{III.8}$$

If the domain shift is purely scaling *in the projected space*, i.e., $g(\alpha z) = \beta g(z)$ for some $\beta > 0$ (a standard abstraction of domain-wise magnitude change), then

$$u(\alpha z) = \frac{\beta g(z)}{\|\beta g(z)\|_2} = \frac{g(z)}{\|g(z)\|_2} = u(z), \tag{III.9}$$

which implies invariance of cosine similarity.

(ii) (Stability) We use the following lemma: for any nonzero vectors $a, b$,

$$\left\| \frac{a}{\|a\|_2} - \frac{b}{\|b\|_2} \right\|_2 \leq \frac{2\|a - b\|_2}{\min(\|a\|_2, \|b\|_2)}. \tag{III.10}$$

Applying it with $a = g(z')$ and $b = g(\alpha z)$ yields

$$\|u(z') - u(\alpha z)\|_2 = \left\| \frac{g(z')}{\|g(z')\|_2} - \frac{g(\alpha z)}{\|g(\alpha z)\|_2} \right\|_2 \leq \frac{2\|g(z') - g(\alpha z)\|_2}{\min(\|g(z')\|_2, \|g(\alpha z)\|_2)}. \tag{III.11}$$

Since $\|g(z')\|_2 \geq \|g(\alpha z)\|_2 - \|g(z') - g(\alpha z)\|_2$ (triangle inequality), we have $\min(\|g(z')\|_2, \|g(\alpha z)\|_2) \geq \|g(\alpha z)\|_2 - \|g(z') - g(\alpha z)\|_2$, which gives (III.6). If $g$ is $L_g$-Lipschitz, then $\|g(z') - g(\alpha z)\|_2 \leq L_g\|z' - \alpha z\|_2 = L_g\|\delta\|_2$, giving (III.7). □

**Lemma III.3** (Normalization difference bound)**.** *For any nonzero* $a, b \in \mathbb{R}^m$*,*

$$\left\| \frac{a}{\|a\|_2} - \frac{b}{\|b\|_2} \right\|_2 \leq \frac{2\|a - b\|_2}{\min(\|a\|_2, \|b\|_2)}.$$

*Proof.* Write

$$\left\| \frac{a}{\|a\|} - \frac{b}{\|b\|} \right\| \leq \left\| \frac{a}{\|a\|} - \frac{b}{\|a\|} \right\| + \left\| \frac{b}{\|a\|} - \frac{b}{\|b\|} \right\| = \frac{\|a - b\|}{\|a\|} + \|b\| \left| \frac{1}{\|a\|} - \frac{1}{\|b\|} \right|.$$

Noting $|\|a\| - \|b\|| \leq \|a - b\|$ and simplifying gives

$$\left\| \frac{a}{\|a\|} - \frac{b}{\|b\|} \right\| \leq \frac{\|a - b\|}{\|a\|} + \frac{\|a - b\|}{\|b\|} \leq \frac{2\|a - b\|}{\min(\|a\|, \|b\|)}.$$

□

## IV Experimental Details

### A. *Datasets*

We provide additional details of the datasets used for open-domain pretraining and downstream evaluation. Following the leakage-control protocol in the main paper, the pretraining datasets and downstream evaluation datasets are strictly non-overlapping at the dataset level.

*1)* ***Pretraining Datasets****:* Our pretraining corpus is assembled from multiple public time-series repositories, including Monash [7], UEA [8], UCR [9], PRSA [10], and PEMS [11]. As summarized in Table IV.1, the corpus covers four macro-domains: *Energy*, *Nature*, *Health*, and *Transport*. These datasets exhibit heterogeneous temporal characteristics, including different sampling resolutions, variable dimensions, seasonalities, and domain-specific dynamics. In total, the pretraining corpus contains 91.87M time points. Here, "time points" denotes the total number of scalar observations after accounting for all variates in multivariate datasets. To better characterize the domain composition of the pretraining corpus, we also reports the total time points in each macro-domain. The corpus is intentionally heterogeneous: *Transport* and *Nature* contribute the largest portions of observations, while *Energy* and *Health* provide complementary temporal patterns with different resolutions and dependency structures. This diversity supports the learning of transferable representations beyond a single domain.

TABLE IV.1: Detailed statistics and domain-level summary of the open-domain pretraining corpus.

| Domain | Dataset | Resolution | Time Points | Domain Points | Ratio | Source |
|---|---|---|---|---|---|---|
| Energy | Australian Electricity Demand | 30 Min | 1,155,264 | 8,552,411 | 9.31% | Monash [7] |
| | Wind | 4 Sec | 7,397,147 | | | Monash [7] |
| Nature | PRSA | Hourly | 4,628,448 | 30,927,982 | 33.66% | PRSA [10] |
| | Sunspot | Daily | 73,924 | | | Monash [7] |
| | Temperature Rain | Daily | 23,252,200 | | | Monash [7] |
| | Saugeen River Flow | Daily | 23,741 | | | Monash [7] |
| | KDD Cup 2018 | Hourly | 2,942,364 | | | Monash [7] |
| | US Births | Daily | 7,305 | | | Monash [7] |
| Health | SelfRegulationSCP1 | – | 3,015,936 | 6,704,256 | 7.30% | UEA [8] |
| | SelfRegulationSCP2 | – | 3,064,320 | | | UEA [8] |
| | PIGCVP | – | 624,000 | | | UCR [9] |
| Transport | Pems03 | 5 Min | 9,382,464 | 45,688,666 | 49.73% | PEMS [11] |
| | Pems04 | 5 Min | 5,216,544 | | | PEMS [11] |
| | Pems07 | 5 Min | 24,921,792 | | | PEMS [11] |
| | Pems08 | 5 Min | 3,035,520 | | | PEMS [11] |
| | Pedestrian Counts | Hourly | 3,132,346 | | | Monash [7] |
| | | | **Total** | **91,873,315** | **100.00%** | – |

*2) **Evaluation Datasets on TSLib**:* For zero-shot and few-shot forecasting, we evaluate CastorTS on the widely used Time-Series-Library (TSLib) benchmark [12]. The benchmark contains seven datasets: *ETTh1*, *ETTh2*, *ETTm1*, *ETTm2*, *Weather*, *Electricity*, and *Traffic*. These datasets cover electricity, weather, and traffic domains, with the number of variates ranging from 7 to 862 and sampling resolutions ranging from 10 minutes to one hour. This benchmark allows us to assess cross-domain transfer under different temporal resolutions, sequence lengths, and multivariate scales. The ETT datasets are collected from the electric power industry and contain two years of measurements from two counties in China, including oil temperature and electricity-load-related features. They are split into hourly subsets, *ETTh1* and *ETTh2*, and 15-minute subsets, *ETTm1* and *ETTm2*. The *Weather* dataset contains 21 meteorological variables collected every 10 minutes. The *Electricity* dataset records hourly electricity consumption from 321 clients. The *Traffic* dataset contains hourly freeway occupancy rates from 862 sensors in the San Francisco Bay Area. For all TSLib datasets, we use a lookback length of $L = 512$ and forecasting horizons $F \in \{96, 192, 336, 720\}$. In the zero-shot setting, the pretrained model is directly evaluated on the target dataset without target-domain training. In the few-shot setting, we use 5% and 10% of the target-domain training split for lightweight adaptation. For CastorTS, DySP-Mamba and PMP are frozen during few-shot adaptation, and only the prediction head is updated with Top-$K$ proxy injection.

TABLE IV.2: Statistics of the TSLib evaluation datasets.

| Dataset | Vars. | Time Points | Resolution | Domain |
|---|---|---|---|---|
| ETTh1 | 7 | 17,420 | Hourly | Electricity |
| ETTh2 | 7 | 17,420 | Hourly | Electricity |
| ETTm1 | 7 | 69,680 | 15 Min | Electricity |
| ETTm2 | 7 | 69,680 | 15 Min | Electricity |
| Weather | 21 | 52,696 | 10 Min | Weather |
| Electricity | 321 | 26,304 | Hourly | Electricity |
| Traffic | 862 | 17,544 | Hourly | Traffic |

*3) **Evaluation Datasets on GIFT-Eval**:* In addition to TSLib, we evaluate CastorTS on GIFT-Eval [13] to further examine its zero-shot generalization ability under a broader benchmark setting. GIFT-Eval is a general time-series forecasting benchmark that covers diverse domains, sampling frequencies, and prediction horizons. Following the official GIFT-Eval protocol, we report two metrics, SMAPE (Symmetric Mean Absolute Percentage Error) and NRMSE (Normalized Root Mean Squared Error), where lower values indicate better forecasting performance. To avoid data leakage, we remove GIFT-Eval configurations

TABLE IV.3: GIFT-Eval datasets for zero-shot evaluation. Frequencies follow the GIFT-Eval aliases: 10S = 10 seconds, 5T/10T/15T = 5/10/15 minutes, H = hourly, D = daily, and W = weekly. All listed datasets are held out from pretraining.

| Dataset | Category | Frequency | Type | Prediction Lengths |
|---|---|---|---|---|
| `jena_weather` (JW) | Nature | 10T, H, D | Multi. | 10T: 48/480/720; H: 48/480/720; D: 30 |
| `solar` (Sol.) | Energy | 10T, H, D, W | Uni. | 10T: 48/480/720; H: 48/480/720; D: 30; W: 8 |
| `m_dense` (MD) | Transport | H, D | Uni. | H: 48/480/720; D: 30 |
| `loop_seattle` (LS) | | 5T, H, D | Uni. | 5T: 48/480/720; H: 48/480/720; D: 30 |
| `sz_taxi` (SZ) | | 15T, H | Uni. | 15T: 48/480/720; H: 48 |
| `bizitobs_application` (BA) | Web/CloudOps | 10S | Multi. | 60/600/900 |
| `bizitobs_l2c` (BL) | | 5T, H | Multi. | 5T: 48/480/720; H: 48/480/720 |
| `bitbrains_service`† (BS) | | 10S | Multi. | 60/600/900 |

that overlap with either our pretraining corpus or the TSLib target datasets. Specifically, datasets such as ETT, Electricity, Jena Weather, KDD Cup 2018, Saugeen River Flow, Temperature Rain, and US Births are excluded because they overlap with the downstream TSLib datasets or the open-domain pretraining corpus used by CastorTS. The final selected non-overlapping GIFT-Eval configurations used for zero-shot evaluation are summarized in Table IV.3.

### *B. Baselines*

We compare CastorTS with representative forecasting baselines from three categories: pretrained time-series foundation models (TSFMs), LLM-adapted time-series models, and task-specific forecasting models. The pretrained TSFMs further include lightweight models and larger Transformer-based models. For pretrained models, we use the official checkpoints and inference protocols whenever available. To avoid data leakage in zero-shot evaluation, we do not report a baseline result when the target dataset is included in its pretraining corpus. For few-shot evaluation, all adaptable models are tuned on the same target-domain training subset under identical lookback lengths and prediction horizons. Task-specific forecasters are trained only on the target-domain training data and do not use external pretraining corpora. Table IV.4 summarizes the public implementations used for the baseline models.

TABLE IV.4: Code repositories for baseline models.

| Model Type | Model | Code Repository |
|---|---|---|
| Lightweight TSFMs | SEMPO | https://github.com/mala-lab/SEMPO |
| | TTM | https://github.com/ibm-granite/granite-tsfm |
| Transformer-based TSFMs | Time-MoE | https://github.com/Time-MoE/Time-MoE |
| | Timer | https://github.com/thuml/Large-Time-Series-Model |
| | Moirai | https://github.com/SalesforceAIResearch/uni2ts |
| | Chronos | https://github.com/amazon-science/chronos-forecasting |
| | TimesFM | https://github.com/google-research/timesfm |
| | MOMENT | https://github.com/moment-timeseries-foundation-model/moment |
| LLM-based Models | Time-LLM | https://github.com/KimMeen/Time-LLM |
| | GPT4TS | https://github.com/DAMO-DI-ML/NeurIPS2023-One-Fits-All |
| | $S^2$IP-LLM | https://github.com/panzijie825/S2IP-LLM |
| Task-specific Forecasters | iTransformer | https://github.com/thuml/iTransformer |
| | DLinear | https://github.com/cure-lab/LTSF-Linear |
| | PatchTST | https://github.com/yuqinie98/PatchTST |
| | TimesNet | https://github.com/thuml/TimesNet |
| | FEDformer | https://github.com/MAZiqing/FEDformer |

*1) **Pretrained Time-Series Foundation Models**:* We include both lightweight TSFMs and larger Transformer-based TSFMs. These models are pretrained on heterogeneous time-series corpora and are mainly designed for zero-shot or few-shot forecasting across diverse domains.

- **SEMPO** [14] is a lightweight time-series foundation model that improves forecasting transferability under constrained model size and pretraining cost.

- **TTM** [15] is a compact mixer-based pretrained forecaster for zero-shot and few-shot multivariate forecasting. It provides several released variants, including $\text{TTM}_B$ (1M), $\text{TTM}_E$ (4M), and $\text{TTM}_A$ (5M). We use the official checkpoint corresponding to the reported setting.
- **Time-MoE** [16] is a Transformer-based TSFM with a mixture-of-experts architecture. By activating only a subset of experts for each token, it increases model capacity while retaining sparse computation. We evaluate its base and large variants, denoted as $\text{Time-MoE}_B$ and $\text{Time-MoE}_L$.
- **Timer** [17] formulates time-series forecasting as generative sequence modeling. It adopts a decoder-only Transformer architecture and is pretrained with a next-token prediction objective over large-scale time-series corpora.
- **Moirai** [18] is a universal forecasting Transformer trained across diverse time-series domains. It is designed for zero-shot forecasting and supports multiple model sizes, including $\text{Moirai}_S$ (14M), $\text{Moirai}_B$ (91M), and $\text{Moirai}_L$ (311M).
- **Chronos** [19] converts continuous time-series values into discrete tokens and trains language-model-style forecasters over the resulting token sequences. We evaluate representative Chronos variants, including $\text{Chronos}_S$ (46M) and $\text{Chronos}_L$ (710M).
- **TimesFM** [20] is a decoder-only time-series foundation model for zero-shot forecasting. It adopts a patch-based input representation and predicts future patches, thereby reducing the number of autoregressive decoding steps.
- **MOMENT** [21] is a family of open time-series foundation models based on patch-wise Transformer encoders. It learns general-purpose time-series representations through masked reconstruction over heterogeneous datasets. We use $\text{MOMENT}_L$ (385M) in our comparison.

*2) **LLM-based Time-Series Models**:* We further compare with LLM-adapted forecasting models, including Time-LLM [22], GPT4TS [23], and $\text{S}^2$IP-LLM [24]. These methods reuse pretrained language models for time-series forecasting by converting numerical sequences into LLM-compatible representations, prompts, or embedding spaces.

- **Time-LLM** [22] reprograms time-series inputs into textual or prototype-based representations and feeds them into an off-the-shelf LLM for forecasting. It supports multiple language-model backbones, including LLaMA- and GPT-2-based variants.
- **GPT4TS** [23] adapts a pretrained GPT-style language model to time-series forecasting. It fine-tunes only a small subset of parameters, such as the input embedding, positional embedding, layer normalization, and output layer, while reusing most pretrained language-model parameters.
- **$\text{S}^2$IP-LLM** [24] improves LLM-based forecasting through semantic-space-informed prompt learning. It aligns time-series embeddings with the semantic space of a pretrained LLM, enabling the language model to better interpret temporal patterns.

*3) **Task-Specific Forecasting Models**:* We also include strong task-specific forecasters, including iTransformer [25], DLinear [26], PatchTST [5], TimesNet [12], and FEDformer [27]. Unlike pretrained TSFMs, these models are trained from scratch on each target dataset and therefore serve as specialized forecasting baselines.

- **iTransformer** [25] inverts the standard temporal-token formulation by treating each variate as a token. This design allows self-attention to directly model cross-variate dependencies.
- **DLinear** [26] is a decomposition-based linear forecaster. It separates trend and seasonal components and applies linear projections from historical observations to future predictions.
- **PatchTST** [5] divides each univariate series into patch tokens and processes channels independently with a shared Transformer backbone, improving the efficiency of long-context forecasting.
- **TimesNet** [12] transforms one-dimensional temporal variations into structured two-dimensional representations according to automatically discovered periods, enabling convolutional modeling of multi-periodic patterns.
- **FEDformer** [27] combines seasonal-trend decomposition with frequency-domain attention, reducing the cost of long-range dependency modeling in long-horizon forecasting.

### *C. Implementation Details*

We implement CastorTS and all trainable baselines in PyTorch. All experiments are conducted on $4\times$ NVIDIA A6000 GPUs with 48GB memory.

*a) **Model Configuration**:* Unless otherwise specified, we set the hidden dimension to $D = 256$ and the proxy dimension to $d = 128$. The default DySP-Mamba configuration contains 18.7M parameters. The feed-forward dimension is set to $d_{\text{ff}} = 256$, and the head dropout is set to 0.2. For dynamic tokenization, the boundary threshold is fixed as $\tau_{\text{tok}} = 0.5$. In the proxy-guided prediction stage, we retrieve the top-$K$ most similar source-domain proxies and set $K = 3$. For FloorAttn, we set the attention-floor coefficient to $\lambda_{\text{floor}} = 0.2$. This choice is guided by Theorem II.1, where proxy aggregation is jointly determined by $\lambda_{\text{floor}}$ and the stacked FloorAttn depth $H$. Specifically, the proxy token is guaranteed to receive the largest incoming rollout attention once

$$H \geq H_{\min} = \left\lceil \frac{\log(1/2)}{\log(1-\lambda_{\text{floor}})} \right\rceil. \tag{IV.1}$$

When $\lambda_{\text{floor}} = 0.2$, we have $H_{\min} = 4$, which ensures stable proxy aggregation with a moderate network depth. Compared with a smaller value such as $\lambda_{\text{floor}} = 0.1$, which requires $H_{\min} = 7$, this setting avoids relying on an overly deep FloorAttn stack. Compared with larger values such as $\lambda_{\text{floor}} = 0.3$ or $0.5$, it also avoids forcing excessive attention mass to the proxy token and preserves sufficient flexibility for token-to-token interactions. Therefore, $\lambda_{\text{floor}} = 0.2$ provides a balanced trade-off between stable domain-proxy aggregation and adaptive multivariate dependency modeling.

*b) **Pretraining**:* CastorTS is pretrained on 91M time points from non-overlapping open-domain datasets. During pretraining, each mini-batch consists of cross-domain MVTS pairs. For each input sample, Causality-Aware Augmentor (CAA) generates one causality-preserved positive view and one causality-disturbed negative view based on the inferred CUTS+ causal graph. The final pretraining objective combines temporal reconstruction, causality-aware contrastive learning, and proxy-manifold alignment:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{temporal}} + \lambda_2 \mathcal{L}_{\text{causal}} + \lambda_3 \mathcal{L}_{\text{proxy}}, \tag{IV.2}$$

where we set $\lambda_1 = 0.2$, $\lambda_2 = 0.5$, and $\lambda_3 = 0.3$. We train CastorTS for 10 epochs with a batch size of 2,048. We use the AdamW optimizer with learning rate $1 \times 10^{-3}$, weight decay 0.1, $\beta_1 = 0.9$, and $\beta_2 = 0.95$. The learning rate is linearly warmed up for the first 10,000 steps and then kept constant.

*c) **Zero-shot Evaluation**:* For zero-shot forecasting on TSLib, we use a lookback length of $L = 512$ and forecasting horizons $F \in \{96, 192, 336, 720\}$ for all datasets. The pretrained model is directly evaluated on each target dataset without target-domain training. The retrieved Top-$K$ source proxies are injected into the prediction head as compact domain priors. The evaluation batch size is set to 32. For GIFT-Eval, we follow the official zero-shot evaluation protocol and report NRMSE and SMAPE.

*d) **Few-shot Adaptation**:* For few-shot forecasting, we use 5% and 10% of the target-domain training split. During adaptation, DySP-Mamba and PMP are frozen, and only the lightweight prediction head is updated. The same lookback length $L = 512$ and horizons $F \in \{96, 192, 336, 720\}$ are used. Following SEMPO [14], we train the prediction head for at most 20 epochs with a batch size of 32 and apply early stopping with a patience of 6 according to the validation loss.

*e) **Baselines and Leakage Control**:* For pretrained foundation-model baselines, we use official checkpoints and official inference protocols whenever available. To avoid data leakage, we omit a baseline result when the target dataset is included in its pretraining corpus and mark the corresponding entry as "-". For trainable baselines, we use the same lookback length, forecasting horizons, data splits, and batch size as CastorTS. When official results under the identical setting are available, we directly use the reported numbers; otherwise, we rerun the baseline using its official implementation.

*f) **Efficiency Measurement**:* For standardized efficiency comparison, we measure per-batch inference cost under input length 512, prediction horizon 96, and batch size 32. GPU latency is measured after warm-up runs using CUDA synchronization, and peak memory is recorded as the maximum allocated GPU memory during inference. All efficiency results are measured under the same A6000 GPU environment.

*g) **Reporting**:* We report mean squared error (MSE) and mean absolute error (MAE) for TSLib experiments, and report NRMSE and SMAPE for GIFT-Eval experiments. All results are averaged over three runs with different random seeds. In each table, the best and second-best results are highlighted in red and underlined, respectively.

### *D. Computational Complexity*

Let $N$ be the number of variates, $L$ the input length, and $D$ the hidden dimension. Transformer-based TSFMs apply self-attention over flattened time-variate tokens, incurring quadratic complexity in $NL$. In contrast, CastorTS uses SSD-based temporal encoding with linear sequence scaling, and applies FloorAttn only after temporal compression at the variate level. As summarized in Table IV.5, the dominant training cost is reduced from $\mathcal{O}((NL)^2 D)$ to $\mathcal{O}(NLD^2 + N^2 D)$, while preserving matrix-multiplication-friendly computation. Thus, CastorTS avoids full token-level self-attention and remains scalable to long multivariate sequences.

TABLE IV.5: Computational complexity comparison. M.M. denotes matrix-multiplication-friendly computation.

| | Transformer TSFMs | SSD+FloorAttn (Ours) |
|---|---|---|
| Training FLOPs | $\mathcal{O}((NL)^2 D)$ | $\mathcal{O}(NLD^2 + N^2 D)$ |
| Inference FLOPs | $\mathcal{O}(NLD)$ | $\mathcal{O}(D^2 + ND)$ |
| Memory | $\mathcal{O}((NL)^2)$ | $\mathcal{O}(NLD + N^2)$ |
| M.M. | ✓ | ✓ |

## V ADDITIONAL EXPERIMENTAL RESULTS

### A. GIFT-Eval Benchmark Results

To further evaluate the zero-shot generalization ability of CastorTS, we conduct additional experiments on the GIFT-Eval benchmark [13]. GIFT-Eval provides a broad evaluation suite for general time-series forecasting across heterogeneous domains, sampling frequencies, and prediction horizons. To avoid data leakage, we remove datasets that overlap with either our pretraining corpus or the TSLib evaluation datasets. After filtering, we retain eight datasets from four domains: *Nature*, *Energy*, *Transport*, and *Web/CloudOps*. Among them, *Nature*, *Energy*, and *Transport* are covered by the macro-domains used in our pretraining corpus, whereas *Web/CloudOps* is an unseen domain for CastorTS. Following the official GIFT-Eval protocol, we report NRMSE and SMAPE, where lower values indicate better forecasting performance.

TABLE V.1: Zero-shot results on the GIFT-Eval benchmark. Lower NRMSE/SMAPE values indicate better prediction. The best result is highlighted in bold and the second-best result is underlined. JW: `jena_weather`; Sol.: `solar`; MD: `m_dense`; LS: `loop_seattle`; SZ: `sz_taxi`; BA: `bizitobs_application`; BL: `bizitobs_l2c`; BS: `bitbrains_service`.

| Model | Nature | | Energy | | Transport | | | | | | Web/CloudOps | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | JW | | Sol. | | MD | | LS | | SZ | | BA | | BL | | BS | |
| | NRMSE | SMAPE | NRMSE | SMAPE | NRMSE | SMAPE | NRMSE | SMAPE | NRMSE | SMAPE | NRMSE | SMAPE | NRMSE | SMAPE | NRMSE | SMAPE |
| CastorTS | **0.226** | **0.629** | **1.062** | **1.203** | **0.311** | **0.175** | **0.159** | **0.129** | **0.367** | 0.409 | 0.201 | 0.144 | 1.102 | 1.152 | 0.269 | 0.182 |
| SEMPO | 0.236 | 0.638 | 1.084 | 1.217 | 0.392 | 0.295 | 0.160 | 0.131 | 0.369 | **0.406** | 0.213 | 0.162 | **0.780** | **0.880** | 0.299 | 0.228 |
| Chronos$_L$ | 0.241 | 0.657 | 1.144 | 1.270 | 0.319 | 0.186 | 0.175 | 0.132 | 0.372 | 0.425 | 0.160 | **0.098** | 1.010 | 1.140 | 0.236 | **0.139** |
| Chronos$_B$ | 0.243 | 0.653 | 1.086 | 1.230 | 0.314 | 0.181 | 0.177 | 0.133 | 0.375 | 0.418 | 0.155 | 0.110 | 1.040 | 1.135 | 0.233 | 0.150 |
| Chronos$_S$ | 0.261 | 0.649 | 1.144 | 1.252 | 0.317 | 0.183 | 0.181 | 0.134 | 0.380 | 0.450 | **0.147** | 0.107 | 0.999 | 1.076 | **0.227** | 0.140 |
| **1st Count** | CastorTS: **9** SEMPO: 3 Chronos$_L$: 2 Chronos$_B$: 0 Chronos$_S$: 2 | | | | | | | | | | | | | | | |

As shown in Table V.1, CastorTS achieves the best performance on 9 out of 16 metrics across 5 out of 8 datasets. Notably, all winning cases of CastorTS come from domains covered during pretraining, namely *Nature*, *Energy*, and *Transport*. On these seen-domain datasets, CastorTS achieves the best result on 9 out of 10 metrics and obtains the second-best result on the remaining metric. It also consistently outperforms all Chronos variants on both NRMSE and SMAPE across the seen-domain datasets, demonstrating strong zero-shot transferability to held-out datasets from related macro-domains.

These results further support the effectiveness of the proposed domain-proxy mechanism. Since the retrieved proxies are learned from open-domain pretraining data, the strong performance on *Nature*, *Energy*, and *Transport* suggests that the learned proxy manifold captures reusable domain-level priors and can provide useful guidance for zero-shot forecasting on related but unseen datasets. In contrast, CastorTS is less competitive on the unseen *Web/CloudOps* domain, where Chronos variants and SEMPO achieve stronger results depending on the dataset and metric. This suggests that proxy quality is bounded by the domain diversity of the pretraining corpus: when the target domain is not covered during pretraining, the retrieved proxies may provide less reliable domain guidance. This observation highlights a promising future direction of expanding the pretraining corpus to broader domains, which may further improve the cross-domain transferability of CastorTS.

### B. Full Results for Ablation Study

We conduct three ablation studies to identify the sources of CastorTS's gains: module-level ablations test component necessity, objective-level ablations isolate each loss term with the architecture fixed, and proxy injection ablations examine whether learned proxies can serve as downstream domain priors.

*1) **Module-level Ablations**:* We first ablate the major components of CastorTS in the zero-shot setting. As shown in Fig. V.1, the full model achieves the lowest MSE on both *Weather* and *ETTh1*, with 0.230 and 0.403, respectively. Replacing DySP-Mamba with vanilla Mamba causes the largest degradation, increasing MSE to 0.503 on *Weather* and 0.743 on *ETTh1*, which confirms the importance of the dynamic symmetric pyramid backbone for multi-scale temporal modeling. Removing the Siamese framework also severely hurts performance, raising MSE to 0.471 and 0.667, showing that paired view-based pretraining is critical for transfer. Removing CAA and PMP leads to smaller but consistent drops: CAA mainly improves causality-aware view construction, while PMP contributes to domain-level proxy alignment. Overall, these results show that CastorTS benefits from the joint design of temporal modeling, causal invariance learning, and proxy-based domain alignment.

*2) **Objective-level Ablations**:* We next ablate the pretraining objectives while keeping all modules unchanged. The full objective is $\mathcal{L} = \lambda_1\mathcal{L}_{\text{temporal}} + \lambda_2\mathcal{L}_{\text{causal}} + \lambda_3\mathcal{L}_{\text{proxy}}$, where $\mathcal{L}_{\text{proxy}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{lap}}$. For each variant, only the corresponding loss term is disabled. As shown in Table V.2, the full objective consistently performs best. Removing $\mathcal{L}_{\text{causal}}$ yields the largest degradation, increasing MSE by 53.0% on *Weather* and 32.0% on *ETTh1*, which indicates that contrastive supervision over causality-aware views is the key driver of transferability. Removing $\mathcal{L}_{\text{temporal}}$ also causes clear drops of 38.3% and 16.9%, confirming the role of reconstruction in preserving fine-grained temporal information. Removing $\mathcal{L}_{\text{proxy}}$ increases MSE by 17.8% and 21.3%, showing that proxy-level alignment helps organize domain-transferable representations. Within $\mathcal{L}_{\text{proxy}}$, removing

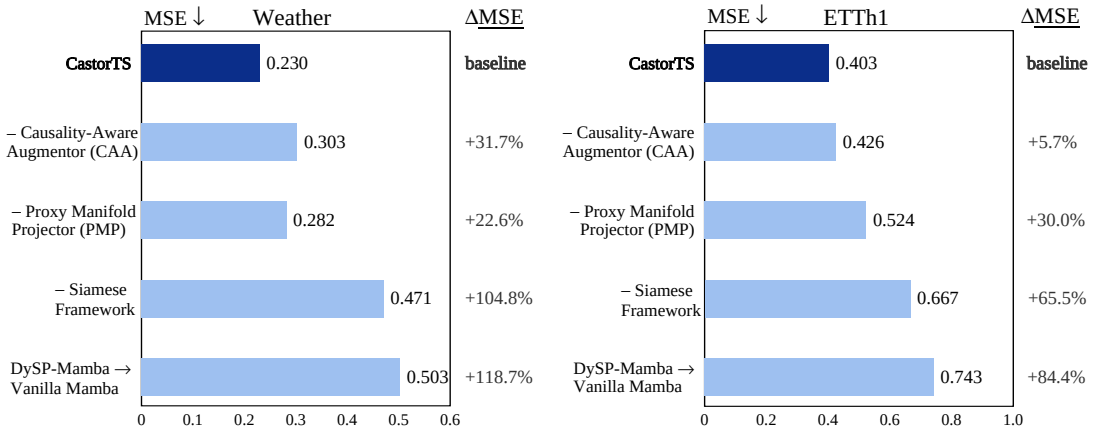


Fig. V.1: Module-level ablations. Bars report MSE, and the right columns show relative MSE degradation over the full model.

$\mathcal{L}_{\text{CE}}$ is more harmful than removing $\mathcal{L}_{\text{lap}}$, suggesting that contrastive proxy alignment provides the main discriminative signal, while Laplacian regularization further smooths the proxy manifold. These results verify that CastorTS benefits from the joint optimization of temporally coherent, causally faithful, and domain-transferable objectives.

TABLE V.2: Objective-level ablations.

| Dataset | Full | w/o $\mathcal{L}_{\text{temporal}}$ | w/o $\mathcal{L}_{\text{causal}}$ | w/o $\mathcal{L}_{\text{proxy}}$ | w/o $\mathcal{L}_{\text{CE}}$ | w/o $\mathcal{L}_{\text{lap}}$ |
|---|---|---|---|---|---|---|
| Weather | **0.230** | 0.318<br>+38.3% | 0.352<br>+53.0% | 0.271<br>+17.8% | 0.259<br>+12.6% | 0.247<br>+7.4% |
| ETTh1 | **0.403** | 0.471<br>+16.9% | 0.532<br>+32.0% | 0.489<br>+21.3% | 0.462<br>+14.6% | 0.439<br>+8.9% |

*3) **Proxy Injection Ablation**:* Finally, we test whether learned proxies are useful during downstream forecasting. We compare the same pretrained model with and without Top-$K$ proxy injection. As shown in Table V.3, proxy injection improves *Weather* in both zero-shot and 10% few-shot settings. In zero-shot forecasting, it reduces MSE/MAE from 0.308/0.338 to 0.230/0.265. In 10% few-shot forecasting, it reduces MSE/MAE from 0.294/0.309 to 0.227/0.260. These results confirm that the learned proxies are not merely auxiliary pretraining variables, but reusable domain priors for lightweight adaptation.

TABLE V.3: Proxy injection ablation.

| Setting | MSE ↓ | | | MAE ↓ | | |
|---|---|---|---|---|---|---|
| | w/o Inj. | w/ Inj. | Relative Gain | w/o Inj. | w/ Inj. | Relative Gain |
| Zero-shot | 0.308 | **0.230** | 25.3% | 0.338 | **0.265** | 21.6% |
| 10% Few-shot | 0.294 | **0.227** | 22.8% | 0.309 | **0.260** | 15.9% |

### C. *Full Results on Graph Corruption*

Since CAA uses an inferred causal graph to construct causality-preserved positive views and causality-disturbed negative views, we further examine whether CastorTS is sensitive to errors in the causal graph. In the main paper, we report the average relative MSE increase under different graph corruptions. Here, we provide the full raw MSE values on *Weather* and *Traffic* under both zero-shot and few-shot settings.

We start from the CUTS+ graph used by default and inject three types of corruption. *Edge dropout* randomly removes a proportion $\rho$ of existing causal edges, simulating missing causal dependencies. *Edge addition* randomly inserts extra edges between originally disconnected variates, simulating false-positive causal links. *Edge rewiring* replaces a proportion $\rho$ of causal edges with random connections, which changes both the edge endpoints and the implied causal semantics. We evaluate three corruption levels, $\rho \in \{10\%, 20\%, 50\%\}$.

TABLE V.4: Full MSE results under causal graph corruption. Lower values indicate better forecasting performance.

| Corruption Type | Noise Ratio $\rho$ | Zero-shot | | Few-shot | |
|---|---|---|---|---|---|
| | | Weather | Traffic | Weather | Traffic |
| **None (ours)** | 0 | **0.230** | **0.458** | **0.227** | **0.414** |
| Edge Dropout | 10% | 0.227 | 0.460 | 0.233 | **0.409** |
| | 20% | 0.233 | 0.468 | 0.239 | 0.412 |
| | 50% | 0.241 | 0.483 | 0.263 | 0.439 |
| Edge Addition | 10% | 0.233 | 0.465 | 0.225 | 0.419 |
| | 20% | 0.241 | 0.463 | 0.244 | 0.415 |
| | 50% | 0.261 | 0.573 | 0.277 | 0.461 |
| Edge Rewiring | 10% | 0.243 | 0.457 | 0.244 | 0.431 |
| | 20% | 0.246 | 0.461 | 0.251 | 0.438 |
| | 50% | 0.305 | 0.613 | 0.282 | 0.598 |

Table V.4 shows that CastorTS remains stable under moderate causal graph noise. When $\rho \leq 20\%$, all three corruption types lead to only small fluctuations in MSE, indicating that CAA does not require perfectly recovered causal graphs to provide useful structural guidance. Among the three corruption types, edge dropout is generally the least harmful, suggesting that missing part of the causal structure can still be tolerated by the Siamese pretraining objective. In contrast, false or semantically incorrect edges are more damaging. Edge addition introduces spurious dependencies, while edge rewiring changes the causal semantics of existing edges. Under severe corruption with $\rho = 50\%$, edge rewiring causes the largest degradation, increasing the MSE to 0.305/0.613 under zero-shot forecasting and 0.282/0.598 under few-shot forecasting on Weather/Traffic. This confirms the observation in the main paper that incorrect or rewired causal edges are more harmful than missing edges. Overall, these results support CAA as a robust causal prior: it benefits from causal structure but is not overly fragile to moderate graph-estimation errors.

## VI Related Work

### A. Time-Series Foundation Models

Time-series foundation models (TSFMs) aim to improve zero- and few-shot forecasting by pretraining transferable forecasters on heterogeneous temporal corpora [28], [29]. A prominent line of work trains time-series-native foundation models from scratch. Encoder-style models such as MOMENT [21] learn general-purpose temporal representations via masked reconstruction, while Moirai [18] trains a universal Transformer forecaster across datasets with diverse frequencies, horizons, and variate dimensions. Decoder-style and generative TSFMs, including TimesFM [20], Lag-Llama [30], Timer [17], Time-MoE [16], TimeGPT [31], Toto [32], and Sundial [33], formulate forecasting as autoregressive or probabilistic sequence generation. Chronos [19] further discretizes continuous observations into tokens and applies language-model-style next-token prediction to time series. Another line of work adapts pretrained large language models (LLMs) to numerical time series. PromptCast [34] and LLMTime [35] serialize numerical observations into textual prompts. GPT4TS [23] reuses pretrained language-model parameters for general time-series analysis, and Time-LLM [22] bridges time-series patches and LLM embeddings through reprogramming. TEMPO [36], $S^2$IP-LLM [24], and AutoTimes [37] further improve LLM-based forecasting via decomposition-aware prompting, semantic alignment, or autoregressive adaptation. These approaches demonstrate the strong transferability of large pretrained forecasters, but their gains often rely on increasing Transformer or LLM capacity and pooling heterogeneous temporal sources in a largely domain-agnostic manner. Such capacity- and data-scaling paradigms can be computationally expensive. In contrast, CastorTS targets efficient TSFM pretraining by improving architectural and representational efficiency, rather than relying solely on large-scale model and data expansion.

### B. Efficient Forecasting Backbones and Lightweight TSFMs

Efficient forecasting backbones reduce computational cost by incorporating time-series-specific inductive biases in place of generic self-attention. DLinear [26] shows that decomposition-based linear forecasting can be highly competitive for long-horizon prediction. iTransformer [25] treats variates as tokens to better model cross-variate dependencies. MLP-based forecasters such as TSMixer [38] and TimeMixer [39] replace quadratic self-attention with lightweight mixing operations. State-space models provide another route to efficient long-range temporal modeling: Damba-ST [40], TimeMachine [41], and MambaTS [42] replace attention with selective state-space transitions, enabling linear-time sequence modeling.

Lightweight TSFMs aim to preserve the transferability of pretrained forecasters while reducing deployment and adaptation costs. One line of work scales down general-purpose TSFMs, as in compact variants of Chronos [19] and Moirai [18], which reduce parameter count and inference cost. Another line designs compact forecasting-specific architectures or economical pretraining pipelines from the outset. For example, TTM [15] builds on the lightweight TSMixer architecture, while SEMPO [14] improves data efficiency through an energy-aware spectral decomposition module. TiRex [43] uses an xLSTM-based recurrent foundation model for efficient in-context forecasting. These studies show that foundation-model forecasting need not depend exclusively on very large backbones or costly inference. However, most existing efficiency gains primarily arise from model compression or lightweight backbone design. CastorTS instead improves efficiency along two dimensions: computational efficiency, achieved by Mamba-style linear-time temporal modeling, and data efficiency, achieved by Siamese contrastive pretraining that constructs informative data pairs from the pretraining corpus.

### *C. Siamese Representation Learning for Time-Series Pretraining*

Self-supervised time-series pretraining learns transferable representations from unlabeled temporal data. Existing methods are commonly built upon masked reconstruction or contrastive learning. Masked reconstruction methods train encoders to recover missing points, segments, or patches from partially observed sequences, as in TST [44], PatchTST [5], Ti-MAE [45], SimMTM [46], and TimeMAE [47]. Contrastive methods construct multiple views of time series and learn transformation-invariant representations. Representative examples include TNC [48], TS2Vec [49], TS-TCC [50], CoST [51], TF-C [52], and SoftCLT [53]. Although effective, these methods typically define pretext tasks through observation-level corruption, reconstruction, or heuristic augmentations such as cropping, jittering, masking, scaling, and spectral perturbation. Siamese representation learning provides a more explicit framework for paired-view modeling. Siamese networks are neural architectures with shared parameters across two or more branches, making them well suited for comparing, matching, or distinguishing paired inputs through a common encoder [54]. This shared-weight design has become a central component in self-supervised representation learning, where paired views are encoded by the same network and optimized to preserve useful invariances or predictive correspondences [55]. TimeSiam [56] makes an important step in this direction by adapting Siamese networks to time-series pretraining. TimeSiam randomly samples past and current subseries from the same time series and trains shared Siamese encoders to model their temporal correlations through a past-to-current reconstruction task. Despite this progress, existing Siamese-style contrastive methods mainly construct paired views within the same series or the same domain. As a result, they primarily learn temporal correlation or augmentation invariance under intra-domain dynamics, while under-exploring transferable structures across heterogeneous domains. CastorTS extends Siamese pretraining from intra-domain temporal correlation modeling to cross-domain structural representation learning. Specifically, CastorTS constructs cross-domain Siamese pairs and generates causality-guided views, so that the shared encoder learns stable cross-domain causal invariances representations.

### *D. Causality-Aware Time-Series Modeling*

Causal and Granger-style structures provide useful inductive biases for multivariate time series because directed inter-variate dependencies can be more stable than raw observations under distribution shifts [57]. Classical Granger causality [58] defines a variable as causal for another variable if its historical values improve prediction of the latter. Neural Granger causality methods such as NGC [59] extend this idea with nonlinear predictors and sparsity constraints, while CUTS [60] and CUTS+ [61] infer temporal causal graphs from irregular or high-dimensional multivariate observations. Causal transfer methods further exploit stable mechanisms to improve generalization across domains. CMTN [62], transferable forecasting under causal mechanisms [63], and latent invariant causal adaptation [64] seek domain-robust predictors by leveraging invariant causal relations. These studies support the view that causal structure can be more transferable than raw temporal observations. However, they typically use causal information for adaptation or transfer after the modeling objective has been defined. CastorTS instead incorporates causal structure directly into TSFM pretraining, aligning Siamese representation learning with structural invariance rather than augmentation-induced temporal similarity alone.

### *E. Cross-Domain Forecasting*

Cross-domain forecasting is important when target-domain supervision is limited. Existing methods improve transferability through source–target adaptation, unified modeling, or retrieval-based knowledge reuse. DAF [65] learns domain-invariant attention features while preserving domain-specific components for target forecasting. UniTS [66] unifies multiple time-series tasks and domains within a single pretrained model, while CrossST [67] transfers spatio-temporal knowledge across urban districts. Retrieval-based forecasting reuses stored source knowledge or pretrained representations for target prediction. ROSE [68] learns a Time Series Register and selects target-relevant register tokens for adaptive transfer, while TimeRAF [69] and TS-RAG [70] retrieve useful temporal knowledge for zero-shot forecasting. These methods improve transferability, but they often require explicit source–target adaptation or retrieve examples and register tokens without explicitly organizing source domains into a compact, causality-aware geometry. CastorTS instead learns a hyperspherical domain-proxy manifold during pretraining and retrieves Top-$K$ source proxies as target-relevant priors. This design enables efficient zero-shot prediction and lightweight few-shot adaptation without full-parameter tuning.

# REFERENCES


[1] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[2] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1474–1487.

[3] M. Harris, S. Sengupta, and J. D. Owens, "Parallel prefix sum (Scan) with CUDA," in *GPU Gems 3*, H. Nguyen, Ed. Addison-Wesley Professional, 2007, pp. 851–876.

[4] T. Dao and A. Gu, "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality," in *Forty-first International Conference on Machine Learning, ICML*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. A. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds. PMLR / OpenReview.net, 2024, pp. 10 041–10 071. [Online]. Available: https://proceedings.mlr.press/v235/dao24a.html

[5] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. [Online]. Available: https://openreview.net/forum?id=Jbdc0vTOcol

[6] S. Hwang, B. Wang, and A. Gu, "Dynamic chunking for end-to-end hierarchical sequence modeling," in *The Eleventh International Conference on Learning Representations, ICLR*. OpenReview.net, 2026.

[7] R. Godahewa, C. Bergmeir, G. I. Webb, R. J. Hyndman, and P. Montero-Manso, "Monash time series forecasting archive," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung, Eds., 2021. [Online]. Available: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/eddea82ad2755b24c4e168c5fc2ebd40-Abstract-round2.html

[8] A. Bagnall, H. A. Dau, J. Lines, M. Flynn, J. Large, A. Bostrom, P. Southam, and E. Keogh, "The uea multivariate time series classification archive, 2018," *arXiv preprint arXiv:1811.00075*, 2018.

[9] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The ucr time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[10] S. Zhang, B. Guo, A. Dong, J. He, Z. Xu, and S. X. Chen, "Cautionary tales on air-quality improvement in beijing," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 473, no. 2205, 2017.

[11] P. J. Bickel, C. Chen, J. Kwon, J. Rice, E. van Zwet, and P. Varaiya, "Measuring traffic," *Statistical Science*, vol. 22, no. 4, pp. 581–597, 2007.

[12] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2d-variation modeling for general time series analysis," in *The Eleventh International Conference on Learning Representations*. OpenReview.net, 2023. [Online]. Available: https://openreview.net/forum?id=ju_Uqw384Oq

[13] T. Aksu, G. Woo, J. Liu, X. Liu, C. Liu, S. Savarese, C. Xiong, and D. Sahoo, "GIFT-Eval: A benchmark for general time series forecasting model evaluation," *arXiv preprint arXiv:2410.10393*, 2024. [Online]. Available: https://arxiv.org/abs/2410.10393

[14] H. He, K. Yi, Y. Ma, Q. Zhang, Z. Niu, and G. Pang, "Sempo: Lightweight foundation models for time series forecasting," *arXiv preprint arXiv:2510.19710*, 2025.

[15] V. Ekambaram, A. Jati, P. Dayama, S. Mukherjee, N. H. Nguyen, W. M. Gifford, C. Reddy, and J. Kalagnanam, "Tiny time mixers (ttms): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series," *Advances in Neural Information Processing Systems*, vol. 37, pp. 74 147–74 181, 2024.

[16] X. Shi, S. Wang, Y. Nie, D. Li, Z. Ye, Q. Wen, and M. Jin, "Time-moe: Billion-scale time series foundation models with mixture of experts," in *The Thirteenth International Conference on Learning Representations, ICLR*. OpenReview.net, 2025. [Online]. Available: https://openreview.net/forum?id=e1wDDFmlVu

[17] Y. Liu, H. Zhang, C. Li, X. Huang, J. Wang, and M. Long, "Timer: Generative pre-trained transformers are large time series models," in *Forty-first International Conference on Machine Learning, ICML*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. A. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds. PMLR / OpenReview.net, 2024, pp. 32 369–32 399. [Online]. Available: https://proceedings.mlr.press/v235/liu24cb.html

[18] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified training of universal time series forecasting transformers," in *Forty-first International Conference on Machine Learning*, 2024.

[19] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, M. W. Mahoney, K. Torkkola, A. Gordon Wilson, M. Bohlke-Schneider, and Y. Wang, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=gerNCVqqtR

[20] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," in *Forty-first International Conference on Machine Learning, ICML*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. A. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds. PMLR / OpenReview.net, 2024, pp. 10 148–10 167. [Online]. Available: https://proceedings.mlr.press/v235/das24c.html

[21] M. Goswami, K. Szafer, A. Choudhry, Y. Cai, S. Li, and A. Dubrawski, "MOMENT: A family of open time-series foundation models," in *Forty-first International Conference on Machine Learning, ICML*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. A. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds. PMLR / OpenReview.net, 2024, pp. 16 115–16 152. [Online]. Available: https://proceedings.mlr.press/v235/goswami24a.html

[22] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P. Chen, Y. Liang, Y. Li, S. Pan, and Q. Wen, "Time-LLM: Time series forecasting by reprogramming large language models," in *The Twelfth International Conference on Learning Representations, ICLR*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=Unb5CVPtae

[23] T. Zhou, P. Niu, L. Sun, R. Jin *et al.*, "One fits all: Power general time series analysis by pretrained LM," *Advances in neural information processing systems*, vol. 36, pp. 43 322–43 355, 2023.

[24] Z. Pan, Y. Jiang, S. Garg, A. Schneider, Y. Nevmyvaka, and D. Song, "Ip-llm: Semantic space informed prompt learning with llm for time series forecasting," in *Forty-first International Conference on Machine Learning*, 2024.

[25] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "iTransformer: Inverted transformers are effective for time series forecasting," in *The Twelfth International Conference on Learning Representations, ICLR*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=JePfAI8fah

[26] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 9, 2023, pp. 11 121–11 128.

[27] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 27 268–27 286.

[28] Q. Ma, Z. Liu, Z. Zheng, Z. Huang, S. Zhu, Z. Yu, and J. T. Kwok, "A survey on time-series pre-trained models," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 12, pp. 7536–7555, 2024.

[29] S. R. K. Kottapalli, K. Hubli, S. Chandrashekhara, G. Jain, S. Hubli, G. Botla, and R. Doddaiah, "Foundation models for time series: A survey," *arXiv preprint arXiv:2504.04011*, 2025.

[30] K. Rasul, A. Ashok, A. R. Williams, H. Ghonia, R. Bhagwatkar, A. Khorasani, M. J. D. Bayazi, G. Adamopoulos, R. Riachi, N. Hassen *et al.*, "Lag-llama: Towards foundation models for probabilistic time series forecasting," *arXiv preprint arXiv:2310.08278*, 2023.

[31] A. Garza, C. Challu, and M. Mergenthaler-Canseco, "Timegpt-1," *arXiv preprint arXiv:2310.03589*, 2023.

[32] B. Cohen, E. Khwaja, K. Wang, C. Masson, E. Ramé, Y. Doubli, and O. Abou-Amal, "Toto: Time series optimized transformer for observability," *arXiv preprint arXiv:2407.07874*, 2024.

[33] Y. Liu, G. Qin, Z. Shi, Z. Chen, C. Yang, X. Huang, J. Wang, and M. Long, "Sundial: A family of highly capable time series foundation models," in *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[34] H. Xue and F. D. Salim, "Promptcast: A new prompt-based learning paradigm for time series forecasting," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 6851–6864, 2024.

[35] N. Gruver, M. Finzi, S. Qiu, and A. G. Wilson, "Large language models are zero-shot time series forecasters," *Advances in neural information processing systems*, vol. 36, pp. 19 622–19 635, 2023.

[36] D. Cao, F. Jia, S. O. Arik, T. Pfister, Y. Zheng, W. Ye, and Y. Liu, "TEMPO: Prompt-based generative pre-trained transformer for time series forecasting," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=YH5w12OUuU

[37] Y. Liu, G. Qin, X. Huang, J. Wang, and M. Long, "Autotimes: Autoregressive time series forecasters via large language models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 122 154–122 184, 2024.

[38] V. Ekambaram, A. Jati, N. Nguyen, P. Sinthong, and J. Kalagnanam, "TSMixer: Lightweight mlp-mixer model for multivariate time series forecasting," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD*. ACM, 2023, pp. 459–469. [Online]. Available: https://doi.org/10.1145/3580305.3599533

[39] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J. Y. Zhang, and J. Zhou, "TimeMixer: Decomposable multiscale mixing for time series forecasting," in *The Twelfth International Conference on Learning Representations, ICLR*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=7oLshfEIC2

[40] R. An, Y. Zhang, Z. Liang, W. Fan, Y. Liang, X. Shang, and Q. Li, "Damba-st: Domain-adaptive mamba for efficient urban spatio-temporal prediction," *arXiv preprint arXiv:2506.18939*, 2025.

[41] M. A. Ahamed and Q. Cheng, "Timemachine: A time series is worth 4 mambas for long-term forecasting," *arXiv preprint arXiv:2403.09898*, 2024.

[42] X. Cai, Y. Zhu, X. Wang, and Y. Yao, "Mambats: Improved selective state space models for long-term time series forecasting," *arXiv preprint arXiv:2405.16440*, 2024.

[43] A. Auer, S. Dooley, M. Roussi, M. J. Zhang, A. Garza Ramirez, T. Czepiel, H. Tercan, P. Seifner, M. Vössing, L. Yang, T. Januschowski, and S. Feuerriegel, "TiRex: Zero-shot forecasting across long and short horizons with enhanced in-context learning," *arXiv preprint arXiv:2505.23719*, 2025.

[44] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2021, pp. 2114–2124.

[45] Z. Li, Z. Rao, L. Pan, P. Wang, and Z. Xu, "Ti-MAE: Self-supervised masked time series autoencoders," *arXiv preprint arXiv:2301.08871*, 2023.

[46] J. Dong, H. Wu, H. Zhang, L. Zhang, J. Wang, and M. Long, "SimMTM: A simple pre-training framework for masked time-series modeling," *arXiv preprint arXiv:2302.00861*, 2023.

[47] M. Cheng, Q. Liu, Z. Liu, H. Zhang, R. Zhang, and E. Chen, "TimeMAE: Self-supervised representations of time series with decoupled masked autoencoders," *arXiv preprint arXiv:2303.00320*, 2023.

[48] S. Tonekaboni, D. Eytan, and A. Goldenberg, "Unsupervised representation learning for time series with temporal neighborhood coding," in *International Conference on Learning Representations*, 2021.

[49] Z. Yue, Y. Wang, J. Duan, T. Yang, C. Huang, Y. Tong, and B. Xu, "Ts2vec: Towards universal representation of time series," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 8, 2022, pp. 8980–8987.

[50] E. Eldele, M. Ragab, Z. Chen, M. Wu, C. K. Kwoh, X. Li, and C. Guan, "Time-series representation learning via temporal and contextual contrasting," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 2021, pp. 2352–2359.

[51] G. Woo, C. Liu, D. Sahoo, A. Kumar, and S. C. H. Hoi, "Cost: Contrastive learning of disentangled seasonal-trend representations for time series forecasting," in *International Conference on Learning Representations*, 2022.

[52] X. Zhang, Z. Zhao, T. Tsiligkaridis, and M. Zitnik, "Self-supervised contrastive pre-training for time series via time-frequency consistency," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[53] S. Lee, T. Park, and K. Lee, "Soft contrastive learning for time series," in *International Conference on Learning Representations*, 2024.

[54] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a Siamese time delay neural network," in *Advances in Neural Information Processing Systems*, vol. 6, 1993.

[55] X. Chen and K. He, "Exploring simple Siamese representation learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 15 750–15 758.

[56] J. Dong, H. Wu, Y. Wang, Y. Qiu, L. Zhang, J. Wang, and M. Long, "TimeSiam: A pre-training framework for siamese time-series modeling," *arXiv preprint arXiv:2402.02475*, 2024.

[57] C. Gong, C. Zhang, D. Yao, J. Bi, W. Li, and Y. Xu, "Causal discovery from temporal data: An overview and new perspectives," *ACM Computing Surveys*, vol. 57, no. 4, pp. 1–38, 2024.

[58] C. W. Granger, "Investigating causal relations by econometric models and cross-spectral methods," *Econometrica: journal of the Econometric Society*, pp. 424–438, 1969.

[59] A. Tank, I. Covert, N. Foti, A. Shojaie, and E. B. Fox, "Neural granger causality for nonlinear time series," *arXiv preprint arXiv:1802.05842*, 2018. [Online]. Available: https://arxiv.org/abs/1802.05842

[60] Y. Cheng, R. Yang, T. Xiao, Z. Li, J. Suo, K. He, and Q. Dai, "CUTS: neural causal discovery from irregular time-series data," in *The Eleventh International Conference on Learning Representations, ICLR*. OpenReview.net, 2023. [Online]. Available: https://openreview.net/forum?id=UG8bQcD3Emv

[61] Y. Cheng, L. Li, T. Xiao, Z. Li, J. Suo, K. He, and Q. Dai, "Cuts+: High-dimensional causal discovery from irregular time-series," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 10, 2024, pp. 11 525–11 533.

[62] Z. Li, R. Cai, K.-S. Chai, H. W. Ng, H. D. Vu, M. Winslett, T. Z. J. Fu, B. Xu, X. Yang, and Z. Zhang, "Causal mechanism transfer network for time series domain adaptation in mechanical systems," *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 2, pp. 1–21, 2021.

[63] Z. Li, R. Cai, T. Z. J. Fu, Z. Hao, and K. Zhang, "Transferable time-series forecasting under causal conditional shift," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 4, pp. 1932–1949, 2024.

[64] R. Cai, J. Huang, Z. Yang, Z. Li, E. Eldele, M. Wu, and F. Sun, "Time series domain adaptation via latent invariant causal mechanism," *arXiv preprint arXiv:2502.16637*, 2025.

[65] X. Jin, Y. Park, D. Maddix, H. Wang, and Y. Wang, "Domain adaptation for time series forecasting via attention sharing," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 10 280–10 297.

[66] S. Gao, T. Koker, O. Queen, T. Hartvigsen, T. Tsiligkaridis, and M. Zitnik, "Units: A unified multi-task time series model," in *Advances in Neural Information Processing Systems*, 2024.

[67] A. Liu and Y. Zhang, "Crossst: An efficient pre-training framework for cross-district pattern generalization in urban spatio-temporal forecasting," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 2935–2948.

[68] Y. Wang, Y. Qiu, P. Chen, K. Zhao, Y. Shu, Z. Rao, L. Pan, B. Yang, and C. Guo, "Towards a general time series forecasting model with unified representation and adaptive transfer," *arXiv preprint arXiv:2405.17478*, 2024.

[69] H. Zhang, C. Xu, Y. Zhang, Z. Zhang, L. Wang, and J. Bian, "Timeraf: Retrieval-augmented foundation model for zero-shot time series forecasting," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 9, pp. 5654–5665, 2025.

[70] K. Ning, Z. Pan, Y. Liu, Y. Jiang, J. Y. Zhang, K. Rasul, A. Schneider, L. Ma, Y. Nevmyvaka, and D. Song, "Ts-rag: Retrieval-augmented generation based time series foundation models are stronger zero-shot forecaster," in *Advances in Neural Information Processing Systems*, 2025.